Measuring the Wrong Ledger:
Pitfalls and a Memory Model for KV Cache Studies

Suraj Sedai*
Department of Computer Science, Mercy University, Dobbs Ferry, NY
surajsedai05@gmail.com

Abstract

Techniques that shrink the KV cache are judged by measuring memory, and a single-GPU harness is the most common place to do it. I report four defects in such a harness. None raised an error, and each produced a confident conclusion that was wrong or unsupported. Two examples: INT8 cache quantization saves almost no memory, and the GPU has no out-of-memory limit. All four had the same cause. The measured peak was a different ledger from the one I was reasoning about. To check the ledger, I derive a closed-form model of peak inference memory from the tensors live at the peak. It predicts a held-out model, GPT-2 small, to within 0.03%. With the ledger corrected, the cache's share of peak memory ranges from 5% to 58% across workloads, and the INT8 saving follows it. Peak memory falls by 2.7% at one end and by 26% at the other, and token capacity under a fixed budget rises by 36%.

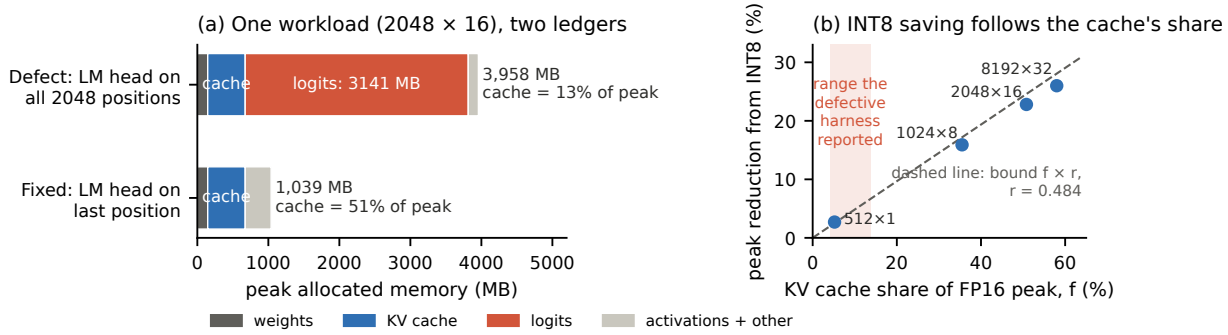


Figure 1: The same INT8 cache, judged against two ledgers. (a) Peak memory at sequence 2048, batch 16 (FP16, decode length 60). Applying the LM head to every prefill position adds a 3141 MB logits tensor and drops the cache's share of peak from 51% to 13%. Cache and weight bytes follow from Eq. (1); the gray segment is the remainder. (b) Peak reduction from INT8 against the cache's share f of FP16 peak. The dashed line is the bound $f \times r$, where $r = 0.484$ is the fraction of cache bytes INT8 removes. The shaded band is the range of cache shares the defective harness reported, where no cache optimization can show a large saving.

1 Introduction

During generation, a transformer keeps the key and value of every previous token in every layer [22]. This KV cache grows with batch size times sequence length, and at long contexts it often decides whether a workload fits in memory at all. Paged allocation [12] and low-precision storage [13] are now standard ways to shrink it.

Whether such a technique helps is settled by measurement. This paper is about how easily that measurement goes wrong. While characterizing KV cache memory on one consumer GPU, I found four defects in my own harness. None raised an error. Each produced a plausible number and a confident conclusion that

*This work was conducted independently, without institutional funding or supervision, on personal hardware.

the corrected harness reverses or shows to be unsupported. Systems research has seen this failure before. A harmless-looking setup choice can invert a result [15], and the remedy is to check a number against the mechanism that should produce it [16].

Figure 1 shows the central example. With the language-model head applied to all 2048 prefill positions, a 3141 MB logits tensor dominated peak memory. The KV cache looked like 13% of peak, and INT8 quantization of the cache looked nearly useless. Applying the head to the last position only raised the cache to 51% of peak and the INT8 saving at that workload to 22.8%. The quantization code did not change. Only the ledger did.

This paper makes three contributions:

- **Four measurement defects**, each with the conclusion it produced, the corrected result, and a signature that reveals it in another harness (Section 3).
- **A closed-form ledger of peak inference memory.** Its activation term comes from the tensors live at the peak rather than a fixed fraction or multiplier. It is accurate to within 0.08% on six architectures, including a held-out model (Section 4).
- **A conditional account of INT8 KV quantization.** Its benefit is bounded by the cache's share of peak, which ranges from 5% to 58% on one model. Under a fixed budget it raises token capacity by 36% (Section 5).

2 Setup

The study model is a GPT-style decoder [22] with 8 layers, 8 heads, $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$ and a 50,257-token vocabulary. It runs in FP16 on one RTX 5070 Ti Laptop GPU with 11.9 GB of memory, under PyTorch 2.11 [17]. The weights are random. That does not affect memory, which depends on tensor shapes, so quality is measured separately on trained GPT-2 models (Section 5.4).

One inference controller drives every run. It calls the same cache interface on the contiguous, paged and INT8 caches, so only the cache changes between runs. A suite of 56 tests runs before any benchmark and checks that the paged cache produces identical tokens and the INT8 cache agrees within quantization tolerance.

Peak memory is `torch.cuda.max_memory_allocated`. Memory budgets are enforced in software with `set_per_process_memory_fraction`, for the reason given in Section 3. KB, MB and GB are powers of two. Memory results reproduce within 1 MB on rerun. Latency is reported but supports no conclusion, because decode at this model size is dominated by per-step overhead. Appendix A gives decode lengths, timing protocol and the remaining details.

3 Four pitfalls

Table 1 summarizes the four defects. Each produced a reasonable-looking number. The last column gives the signature that would catch the same defect in another harness.

LM head over every prefill position. The language-model head ran on all prefill positions instead of the last one. At sequence 2048 and batch 16, this created a 3141 MB logits tensor, 79% of the 3958 MB peak, for 2047 positions that are never sampled. Slicing to the last position cut peak to 1039 MB (Figure 1a). Comparing the measured peak with Eq. (1) would have exposed the gap at once.

A per-sequence loop in the cache interface. The first interface handled one sequence at a time, so it issued batch $\times$ n_{layers} cache calls per token instead of n_{layers}. Latency grew 4.7$\times$ from batch 1 to 16 but only 3.5%

Table 1: Four measurement defects, what each made me conclude, and how to detect it.

Defect	Conclusion it produced	Corrected result	Detection signature
LM head on all prefill positions	Cache is 13% of peak; INT8 saves almost nothing	Cache is 50.8–58.0% of peak at large cells; INT8 cuts peak by 22.8–26.0%	Measured peak exceeds the hand-computed ledger (Eq. 1) by an unexplained term
Per-sequence loop in the cache interface	Decode is launch-bound on this GPU	Right conclusion, wrong evidence: the number measured the harness loop	Latency grows with batch (4.7×) but not with sequence length
Contiguous reservation set to the known final length	Contiguous never fragments	Waste is 0.0 by construction, so the comparison is vacuous	Fragmentation is exactly zero across a whole sweep
Windows CUDA system-memory fallback	The GPU has no out-of-memory limit	14,008.6 MB allocated on a 12,226.6 MB card	A run that should fail, or an allocation larger than the card

from sequence 128 to 2048. It tracked Python loop iterations, not attention work. I read it as evidence that decode is launch-bound. That is true at this model size (Appendix A), but this measurement could not have shown it.

Reservation set to the exact final length. The contiguous cache reserved exactly the known final sequence length, so its reservation waste was 0.0 in every cell. Over-reservation under unknown output length is the reason paging exists [12]. The setup removed the effect under study and then reported that it was absent.

The driver's system-memory fallback. On Windows, NVIDIA's CUDA system-memory fallback is on by default. An allocation larger than the GPU spills into host RAM over PCIe instead of failing. Without a cap, sequence 8192 at batch 64 completed with 14,008.6 MB allocated on a 12,226.6 MB card. A software cap restores a real out-of-memory failure, and a budget stated in software reproduces on any device.

The four defects share one failure. Each number looked reasonable, and the reasoning from it was sound. The conclusion was still wrong or unsupported, because the harness measured a different ledger from the one I was reasoning about. Appendix C gives the full numbers. Appendix E withdraws four claims from earlier versions of this work.

4 The right ledger: a closed-form memory model

Peak depends only on total tokens. In FP16 at decode length 8, sequence 8192 at batch 32 peaks at 7086.5 MB. Sequence 16384 at batch 16 holds the same 262,144 tokens and peaks at 7090.6 MB, only 0.058% higher. This holds because `scaled_dot_product_attention` uses a memory-efficient backend that never builds the batch × heads × seq^2 score matrix [6, 7]. The MATH backend, which does build it, needs 4,888 MB of workspace at sequence 2048 and batch 16, against 32 MB.

The per-token cost has two terms. Peak memory is the weights plus a small allocator overhead plus a per-token cost:

$$c_{\text{token}} = \Big(\underbrace{2\, n_{\text{layers}}\, n_{\text{kv}}\, d_{\text{head}} \cdot \tfrac{S}{L}}_{\text{KV cache}} + \underbrace{3\, d_{\text{model}} + 2\, d_{\text{ff}}}_{\text{activations}} \Big) \times \text{bytes}, \qquad S = \left\lceil \tfrac{L+D}{P} \right\rceil P, \tag{1}$$

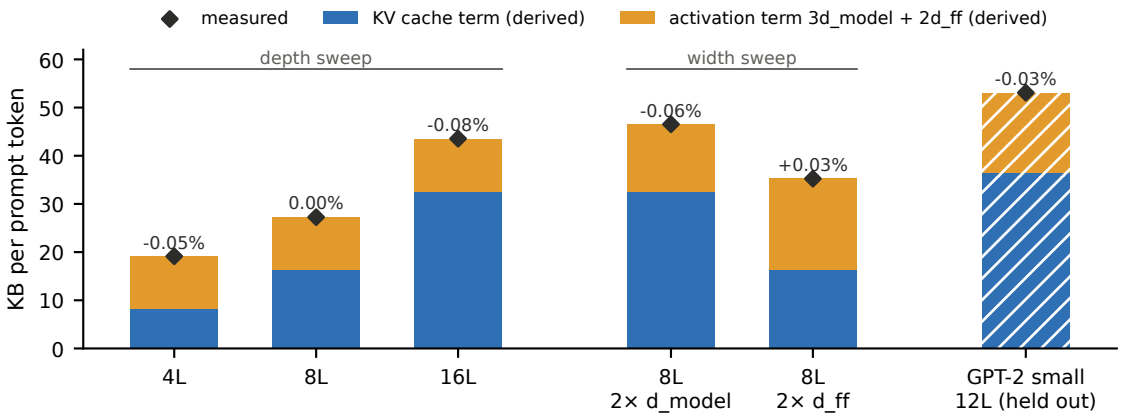


Figure 2: Eq. (1) against measurement. Bars are the two derived terms (sequence 1024, decode length 8); diamonds are measured per-token memory; labels give the prediction error. The cache term doubles with depth while the activation term stays flat. Hatched: GPT-2 small, held out. Full numbers in Table 2.

for L prompt tokens, D decode tokens and P-token cache blocks.

The **cache term** is kept for every layer. It counts the S slots allocated per sequence, not the L prompt tokens, because the pool rounds the prompt plus decode tokens up to whole blocks. For the study model that is 16.25 KB per token at $L = 1024$. Counting prompt tokens instead leaves errors of 0.68–1.23% (Appendix B).

The **activation term** does not grow with depth. Under `torch.no_grad`, a layer's intermediates are freed before the next layer allocates, so only one layer's working set is live at the peak. That peak is at the feed-forward GELU. Five per-token tensors are live there: three of width d_{model} (block input, post-attention residual, second LayerNorm output) and two of width d_{ff} (first projection and its GELU output). A census of live CUDA tensors at the peak finds exactly these five in all six architectures. The standard activation formula multiplies a per-layer cost by depth [11]. That is correct for training, which keeps activations for the backward pass, but it does not carry over to inference.

Accuracy. Figure 2 compares Eq. (1) with measurement on five sweep architectures and one held-out model. Across depth, the activation term stays at 11.00–11.04 KB per token while the cache term doubles. GPT-2 small [19] was not used to build the model. It is predicted at 53.062 KB per token against 53.078 measured, an error of 0.03%. The largest error across all six is 0.08%.

5 What INT8 actually buys

5.1 The cache's share sets the benefit

INT8 stores one byte per element plus one FP16 scale per token and head. At head dimension 64 that keeps 33/64 of the cache bytes, so it removes $r = 48.4\%$ in every cell. The measured ratio matches 33/64 to five decimal places. The effect on peak memory is not fixed. Peak falls by 2.7% at 512×1, 15.9% at 1024×8, 22.8% at 2048×16 and 26.0% at 8192×32 (Figure 1b).

This is Amdahl's law [3] applied to memory. If a term makes up a share f of the total, removing a fraction r of it cuts the total by at most $f \times r$. What the measurements add is the value of f. On one unchanged model it runs from 5.2% to 58.0%, so the same technique is nearly worthless at one end and substantial at the other. In the 19 cells above the smallest, the measured saving reaches 89.6–93.4% of the bound. The shortfall is consistent with the unfused read path briefly holding an INT8 and an FP16 copy (Appendix D).

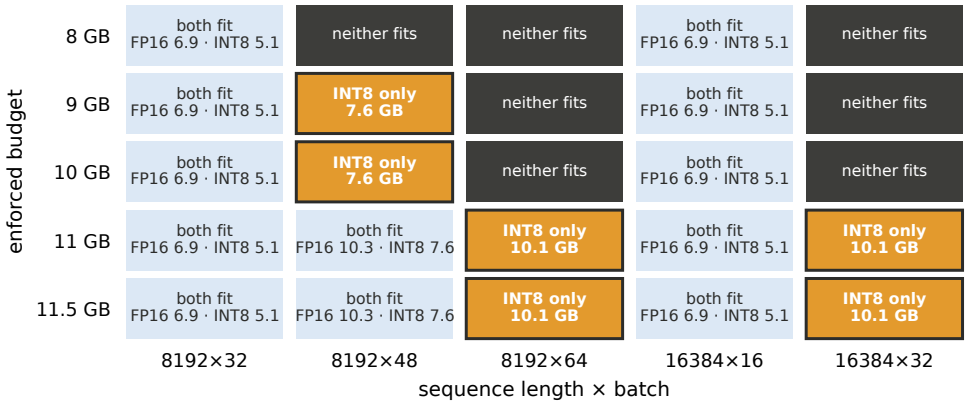


Figure 3: Which workloads fit under each enforced budget (decode length 8). Values are peak allocated memory in GB. In the six outlined cells FP16 runs out of memory and INT8 completes.

The same arithmetic explains the earlier, wrong conclusion. The defective harness put the cache at 4.1–13.8% of peak (the shaded band in Figure 1b). The bound was therefore under 7%, and INT8 delivered a few percent.

5.2 The out-of-memory frontier moves

Under budgets enforced in software, INT8 completes six workloads that FP16 cannot (Figure 3). At 9 and 10 GB it runs 8192×48. At 11 and 11.5 GB it runs 8192×64 and 16384×32, which need 14.0 GB in FP16 and 10.1 GB in INT8. The frontier moves in both directions, toward larger batches and toward longer contexts. The cap limits reserved memory while peak is reported as allocated, so the usable ceiling sits 4.9–13.4% below the nominal budget at 8 GB (Appendix A).

5.3 Capacity, not megabytes

Both precisions are linear in total tokens, with intercepts only 4 MB apart. The slopes are 27.1526 KB per token in FP16 and 19.9253 KB in INT8. INT8 therefore fits 36.3% more tokens in any budget: 413k against 303k at 8 GB, and 623k against 457k at 12 GB.

5.4 Quality cost

On WikiText-2 [14] (284,614 tokens), INT8 changes perplexity by +0.002% on GPT-2 and +0.006% on GPT-2-medium. Perplexity hides something, though. Between 1.2% and 1.5% of next-token argmax decisions flip, and in free-running generation those flips compound, so INT8 changes outputs even where perplexity is intact (Appendix D). Larger models develop channel outliers that make quantization harder [8, 25], so this result holds only up to 355M parameters.

6 Limitations

One architecture family. Every model tested uses multi-head attention and a GELU feed-forward layer. Grouped-query attention [2, 20] shrinks the cache term by the grouping factor, and SwiGLU [21] adds a third d_{ff}-wide tensor. The method of counting tensors live at the peak should transfer, but the coefficients must be

derived again. This is the most serious limit. The closest comparable study evaluates Llama-3.1-8B-Instruct and Mistral-7B-Instruct-v0.3, which use both [1].

One device and platform. The frontier is enforced in software because the Windows fallback hides the physical limit. On Linux the allocator may behave differently. Capacities above 11.9 GB would be extrapolations.

Paging is measured without a fused kernel. The paged read copies a sequence's blocks into a contiguous tensor before attention. The workload also runs uniform-length sequences in lockstep, so paging's benefit cannot appear here (Appendix D).

Latency and quality. GPU clocks are not locked and decode is overhead-bound, so no conclusion rests on latency. Quality is verified only up to 355M parameters.

7 Related work

Measurement methodology. Mytkowicz et al. show that setup choices can invert a conclusion, and found that none of 133 surveyed systems papers adequately accounted for it [15]. This paper applies Ousterhout's advice to measure one level deeper [16] and Hoefler and Belli's rules for reporting noisy results [9]. Two of the four defects, the exact reservation and the driver fallback, are setup choices of the kind Mytkowicz et al. describe. The other two are code defects with the same symptom.

KV cache management. PagedAttention allocates the cache on demand in fixed-size blocks inside vLLM [12], alongside iteration-level batching [26]. vAttention keeps the cache contiguous in virtual memory and argues that paging's indirection costs performance [18]. The paging gap in this harness comes at least partly from an unfused gather, not from that indirection.

KV cache quantization. KIVI and KVQuant quantize keys per channel and values per token to reach 2 bits [10, 13]. The 8-bit per-token scheme here is simpler. Agrawal and Mayer find that realized compression ratio predicts end-to-end performance poorly across mechanisms [1]. This paper holds the mechanism fixed and shows that the same INT8 scheme saves 2.7% or 26% depending only on the cache's share of peak.

Memory estimation. The per-token size of the KV cache is standard in capacity planning [5]. Activation memory is usually estimated as a multiplier on weights, for example total inference memory at about 1.2 times model memory [4], or with the per-layer training formula [11]. Eq. (1) gives an inference-specific term that is flat in depth.

8 Conclusion

Partway through this project, my measurements had told me three things: INT8 barely saves KV cache memory, my GPU has no out-of-memory limit, and decode on it is launch-bound. Two were wrong. The third was right, but not for the reason the number gave. Each came from a harness that measured something other than what I was reasoning about.

The check is cheap. Write down the ledger the peak should contain, compute it from the architecture, and compare. With the ledger right, peak inference memory has a closed form accurate to 0.08%, and the INT8

saving is bounded by the cache's share of peak. A technique pays off only when the resource it saves is the one filling memory.

Code and data availability. Code, raw results with environment records, and scripts that regenerate every figure and table are at https://github.com/Suraj-Sedai/kv-cache-memory.

Use of AI tools. Claude (Anthropic) was used to draft and edit the text and to write analysis and plotting code, all under the author's direction. The research questions, experimental design decisions and interpretation are the author's. The author ran all experiments, verified every reported number against the committed results, and is responsible for all content.

References

[1] N. Agrawal and R. Mayer. Benchmarking KV-cache optimizations across task quality and system performance for long-context serving. arXiv:2607.05399, 2026.

[2] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023. arXiv:2305.13245.

[3] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967. doi:10.1145/1465482.1465560.

[4] Q. Anthony, S. Biderman, and H. Schoelkopf. Transformer math 101. EleutherAI Blog, 2023. https://blog.eleuther.ai/transformer-math/.

[5] J. Austin et al. How to scale your model: All about transformer inference, 2025. https://jax-ml.github.io/scaling-book/inference/.

[6] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024. arXiv:2307.08691.

[7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022. arXiv:2205.14135.

[8] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, 2022. arXiv:2208.07339.

[9] T. Hoefler and R. Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *SC '15*, 2015. doi:10.1145/2807591.2807644.

[10] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *NeurIPS*, 2024. arXiv:2401.18079.

[11] V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro. Reducing activation recomputation in large transformer models. In *MLSys*, 2023.

[12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023. arXiv:2309.06180.

[13] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *ICML*, 2024. arXiv:2402.02750.

[14] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. In *ICLR*, 2017. arXiv:1609.07843.

[15] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney. Producing wrong data without doing anything obviously wrong! In *ASPLOS '09*, pages 265–276, 2009. doi:10.1145/1508244.1508275.

[16] J. Ousterhout. Always measure one level deeper. *Communications of the ACM*, 61(7), 2018.

[17] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[18] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar. vAttention: Dynamic memory management for serving LLMs without PagedAttention. In *ASPLOS '25*, 2025. arXiv:2405.04437.

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.

[20] N. Shazeer. Fast transformer decoding: One write-head is all you need. arXiv:1911.02150, 2019.

[21] N. Shazeer. GLU variants improve transformer. arXiv:2002.05202, 2020.

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[23] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[24] T. Wolf et al. Transformers: State-of-the-art natural language processing. In *EMNLP System Demonstrations*, 2020.

[25] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *ICML*, 2023. arXiv:2211.10438.

[26] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In *OSDI*, 2022.

A Experimental details

Harness and equivalence gate. The cache interface is `write`, `read`, `advance` and `free`, applied to the whole batch at once. The equivalence suite checks that contiguous and paged caches emit identical token sequences for block sizes 1 through 16 and across page boundaries. The INT8 cache must match the contiguous baseline within a tolerance derived from the quantization step and the FP16 rounding of the stored scale.

Memory accounting. Cache memory counts the pages held by live sequences, not the preallocated pool, which would overstate the paged footprint. Fragmentation is waste over total allocation. For the contiguous cache the allocation is its reservation; for the paged cache it is the pages held. Each result row records which basis applies.

Reserved versus allocated memory. `set_per_process_memory_fraction` caps reserved memory, while peak is reported as allocated memory, and the caching allocator reserves in blocks above what is allocated. The usable ceiling is therefore below the stated budget, by 4.9–13.4% at 8 GB. INT8 at 8192×48 fails under an 8 GB budget despite a measured peak of 7791 MB, while FP16 at 16384×16 completes under it at 7091 MB.

Decode length. Decode length is 60 tokens in the precision sweep (Sections 5.1, 5.3, 5.4) and 8 tokens in the architecture sweep (Section 4) and the frontier runs (Section 5.2). Because the cache holds decode positions and rounds up to 16-token blocks, the two settings give different peaks for the same shape. At 8192×32 the sweep holds 8256 slots per sequence and the frontier runs hold 8208. That is 24 MB of cache, which accounts for 24 of the 26 MB between the 7113 MB and 7086.5 MB peaks reported for this shape.

Timing. Each cell runs one timed warmup, used to skip cells that thrash, then two more warmups and 20 measured runs. Per-token latency is the median across runs of each run's median across generated tokens. Out-of-memory and over-budget cells are recorded as outcomes, not dropped.

Why decode is overhead-bound. At the measured matmul throughput of 50.8 TFLOP/s, a decode step's arithmetic would take 0.03–0.65% of its measured latency. Reading the 146 MB of weights and the live cache at the specified 672 GB/s would take 3–15%, and at most a third even counting the extra copy made by the paged gather. The operating point sits below both roofline ceilings [23], so per-token latency reflects kernel launches and host-side Python. After the loop fix, paged decode takes 8.9–11.2 ms per token across the grid, with no trend in batch or sequence length. GPU clocks are not locked. Two runs of one configuration on one commit differed by $1.18\times$ in throughput, so latency supports no conclusion in this paper.

B Memory model details

Baseline fit. For the baseline model, a least-squares fit over four token counts gives 27.25 KB per token and an intercept of 161 MB, with a maximum residual of 2.42 MB against peaks of 1.5–4.5 GB. The intercept is 146 MB of FP16 weights plus about 15 MB of allocator overhead. The two configurations at 524,288 tokens, run without a cap, peak at 14,008.6 and 14,008.7 MB.

Two checks on the activation term. First, solving the three width variants for a d_{model} coefficient, a d_{ff} coefficient and an intercept gives 3.03, 2.00 and -0.018 KB. With three equations in three unknowns this

Table 2: Per-token memory, predicted against measured (KB per prompt token; sequence 1024, decode length 8). Predicted activation is $(3d_{\text{model}} + 2d_{\text{ff}}) \times 2$ bytes. Measured activation is the residual after subtracting the derived cache term.

Architecture (n_{layers} / d_{model} / d_{ff})	Cache	Act. pred.	Act. meas.	Total pred.	Total meas.	Error
4 / 512 / 2048	8.125	11.00	11.01	19.125	19.134	−0.05%
8 / 512 / 2048	16.25	11.00	11.00	27.250	27.250	0.00%
16 / 512 / 2048	32.50	11.00	11.04	43.500	43.536	−0.08%
8 / 1024 / 2048	32.50	14.00	14.03	46.500	46.530	−0.06%
8 / 512 / 4096	16.25	19.00	18.99	35.250	35.238	+0.03%
GPT-2 small, held out: 12 / 768 / 3072	36.5625	16.50	16.52	53.062	53.078	−0.03%

is a consistency check, not an estimate, and it matches the integers and zero. Second, a census of live CUDA tensors at the forward-pass peak finds exactly the five predicted tensors in all six architectures and matches the predicted activation memory within 0.14 MB at 8192 tokens (`results/activation_census/activation_census.json`).

When the GELU is not the peak. The attention phase holds $7d_{\text{model}}$ per token. The GELU is the peak only because $d_{\text{ff}} \geq 2d_{\text{model}}$ in every architecture tested, so the general activation term is $\max(7d_{\text{model}},\ 3d_{\text{model}} + 2d_{\text{ff}})$.

Slot rounding. Figure 4b shows the error when the cache term counts prompt tokens instead of allocated slots: -0.68% to -1.23%, against at most 0.08% with slots. At $L = 512$ the slot count is $\lceil 520/16 \rceil \cdot 16 = 528$, so the cost per prompt token rises to 16.5 KB.

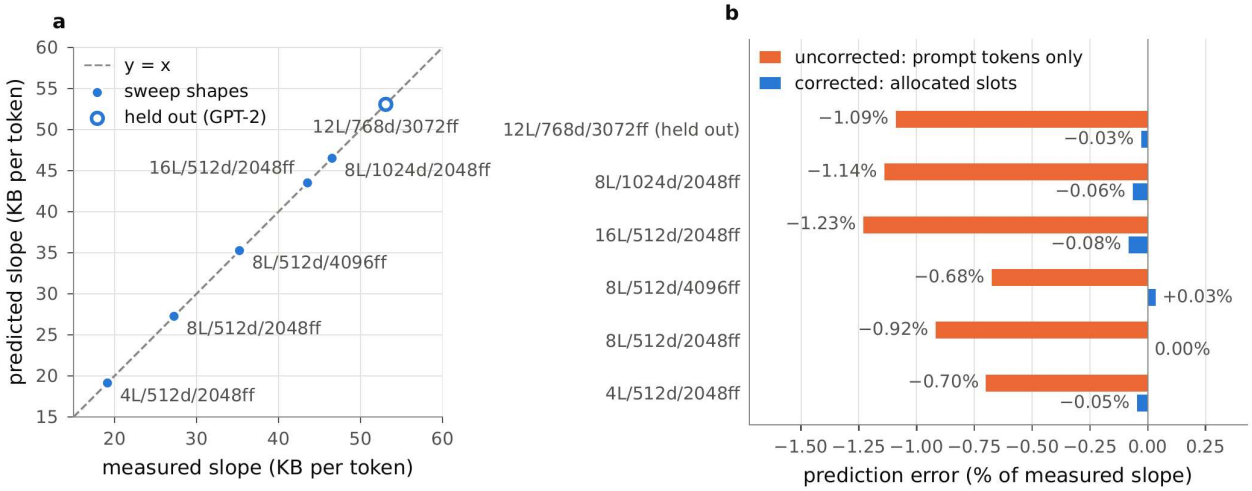


Figure 4: (a) Predicted against measured per-token memory for six architectures. (b) Prediction error when the cache term counts allocated slots (corrected) versus prompt tokens only (uncorrected). Source: `results/arch_sweep/arch_sweep.json`.

Attention backend. This PyTorch build was not compiled with FlashAttention (`results/sdpa_backend_check/sdpa_backend_check.json`). The EFFICIENT and CUDNN backends apply the same principle of never materializing the score matrix [6, 7], and the invariance to batch versus sequence length depends on it.

C Pitfall details

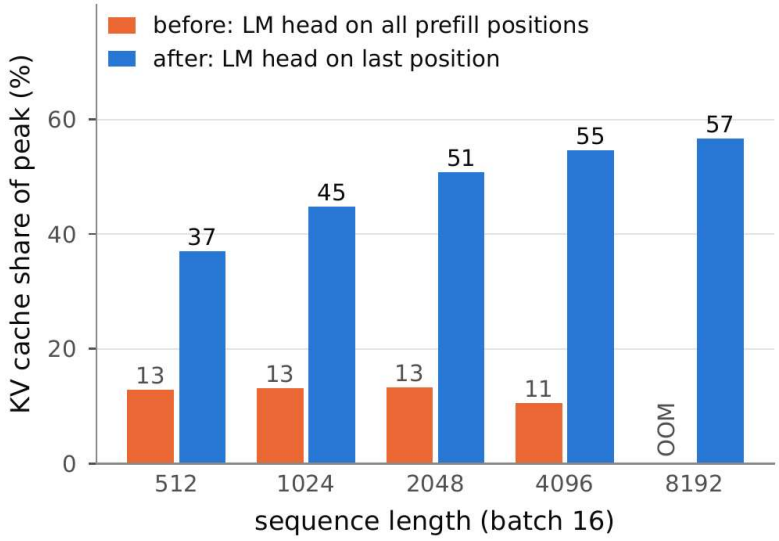


Figure 5: KV cache share of peak, FP16 paged cache, batch 16, before and after the LM head fix. Before: 10.5–13.3% at sequence 512–4096, and 8192 runs out of memory. After: 37.0–56.7% at 512–8192. Sources: `results/pre_fix/exp3_precision_sweep_017b63a.csv`, `results/precision_sweep/exp3_precision_sweep.csv`.

LM head. Figure 5 shows the cache share across sequence lengths at batch 16. Across the full pre-fix grid the cache was 4.1% (512×1) to 13.8% (512×32) of peak.

Per-sequence loop. Paged decode took 4.920 ms per token at batch 1 and 23.228 ms at batch 16 (sequence 128), but 4.920 and 5.093 ms at sequences 128 and 2048 (batch 1). The batch effect of 4.7× exceeds the largest run-to-run drift seen between unpaired sweeps, 1.76×.

Driver fallback. The uncapped run is recorded in `results/pre_fix/wddm_spill.json`.

D INT8 and paging details

Byte ratio. The predicted INT8/FP16 cache ratio is $(1+2/d_{\text{head}})/2 = 33/64 = 0.515625$. The measured ratio is 0.51562 in every cell at batch 16 and above. Smaller cells range from 0.51556 to 0.51588 because cache memory is recorded to 0.01 MB.

Read-path shortfall. Across the 19 cells above the smallest, the realized peak reduction is 89.6–93.4% of the nominal cache saving (median 92.5%): 89.6–91.5% at batch 1 and 90.9–93.4% at batch 8 and above. At 512×1 peak falls by 4.57 MB against a cache saving of 4.36 MB, but a rerun moved its INT8 peak by 1 MB, so that ratio is within noise. The gather returns an INT8 tensor that is then converted to FP16, so both are briefly live. A fused dequantize-in-attention kernel would remove the copy. Confirming this mechanism would need per-phase allocator snapshots.

Frontier peaks. Run without a cap, 8192×64 and 16384×32 peak at 14,008.6 and 14,008.7 MB in FP16 and at 10,328.9 and 10,332.8 MB in INT8. INT8 at 8192×48 peaks at 7791 MB.

Capacity fits. Over the 20 cells of the precision sweep, FP16 fits 27.1526 KB per token with a 168.0 MB intercept, and INT8 fits 19.9253 KB per token with 164.0 MB. These differ from the baseline fit in Appendix B because the sweep mixes sequence lengths and runs at decode length 60. The capacity gain is the slope ratio, $27.1526/19.9253 = 1.3627$. Extrapolated, the fits give 1538k against 2097k tokens at 40 GB and 3083k against 4202k at 80 GB. These are not measurements and hold only if the linear form holds at those sizes.

Quality protocol. GPT-2 and GPT-2-medium [19] run in FP16 through HuggingFace Transformers [24]. The INT8 cache is a `DynamicCache` subclass using the same per-token symmetric scheme as the study cache: a scale of amax/127 over the head dimension, computed in FP32 and stored in FP16. Negative log-likelihood is accumulated with teacher forcing over non-overlapping 512-token windows of the WikiText-2 test set, with a fresh cache per window. Each position's scale depends only on its own vector, so one forward pass gives the exact INT8-cache perplexity. The two caches pick the same next token at 98.54% of positions on GPT-2 and 98.83% on GPT-2-medium. If flips were independent, a 100-token greedy continuation would match FP16 end to end only 23% of the time on GPT-2 and 31% on GPT-2-medium.

Paged versus contiguous. In five interleaved pairs per cell at 512×1, 2048×1 and 2048×16, the median paged/contiguous throughput ratio is 0.58–0.65, paged is slower in all 15 pairs, and peak memory differs by less than 1% (`results/paired_layout_check/paired_layout_check.json`). Timing every `read()` with CUDA events in separate runs (`results/gather_cost/gather_cost_r15.json`) bounds the copy's share. At batch 1, `read()` covers 0.90 of the 1.37 ms gap at 512×1 and 1.21 of 2.74 ms at 2048×1. At 2048×16 the timed interval exceeds the gap, and instrumentation itself moves latency by 0.89–1.15×, so the two cannot be separated there.

E Withdrawn claims

Four claims from earlier versions of this work do not hold up.

Small blocks maximize throughput. A block-size sweep ranked blocks of 8 and 16 first in all 15 cells, by 1.2–1.8× in mean throughput. It ran before the loop and LM head fixes, so it largely measured the harness. Rerun after both fixes, no block size leads in more than 5 of 15 cells, and block 8 is the slowest on average. The 1.17× spread between block-size means equals the 1.18× spread between two runs of one cell, so it supports no ranking [9]. The trade-off does hold: fragmentation rises with block size in all 15 cells, and latency is higher at block 8 than at block 64 in 13.

Contiguous allocation fragments less than paged. With the reservation set to the known final length, contiguous waste is 0.0 by construction. A real comparison needs a reservation policy for unknown output lengths, which this harness does not implement.

Paged overtakes contiguous somewhere in the grid. No crossover appears in 20 cells spanning sequence 128–2048 and batch 1–16. Paged throughput is lower in all 20 unpaired cells and all 15 interleaved pairs, and peak memory agrees within 0.9%. The grid cannot produce a crossover, because uniform-length sequences in lockstep never let paging's flexibility matter.

INT8 does not shift the out-of-memory frontier. This came from the LM head defect, which put the cache at 4.1–13.8% of peak. With the ledger corrected, the cache reaches 58% of peak and the frontier moves (Figure 3).